Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets

Team Leader: SOFIYA NUSRATH J

Team member: ABINAYA S
Team member: VASUNDRADEVI N
Team member: SHAPHARMHATEE J
Team member: MEENAKSHI R

1. Identification Phase

The identification phase defines the problem and determines the need for a solution. In ServiceNow-based environments, organizations often require certain fields to be mandatory only under specific conditions. However, enforcing mandatory fields through scripting can increase complexity and maintenance overhead.

To address this issue, UI Policies are used to dynamically control form behavior such as making fields mandatory, visible, or read-only without writing complex scripts. Additionally, configuration changes must be moved between ServiceNow instances (Development, Testing, Production). Update Sets provide a controlled mechanism to migrate these changes.

This project focuses on implementing mandatory field enforcement using UI Policies and migrating those configurations using Update Sets.

2. Requirement Analysis

During this phase, system requirements and functional needs are analyzed.

Functional Requirements

- The system should make specific fields mandatory based on defined conditions.
- The mandatory rule should activate automatically when the condition is met.
- The configuration should be easily maintainable without custom scripting.
- The implemented changes must be transferable between instances using Update Sets.

Non-Functional Requirements

- The solution should be simple to configure and maintain.
- Changes must not affect existing workflows or system performance.
- Migration should be consistent and error-free.

Tools Required

- ServiceNow Platform
- Admin or Developer Access
- Update Set Management
- Incident or Change Management module for demonstration

3. Project Planning Phase

In this phase, planning is done for implementing the project in a structured manner.

Project Objectives

- Implement mandatory field validation using UI Policies.
- Ensure the configuration works dynamically based on conditions.
- Capture all configuration changes in an Update Set.
- Successfully migrate the changes from Development instance to another instance.

Project Scope

The project focuses on:

- Creating UI Policies
- Configuring UI Policy Actions
- Testing mandatory field behavior
- Migrating configurations using Update Sets

4. Project Development Phase

This phase involves actual implementation in the ServiceNow environment.

Step 1: Create an Update Set

1. Navigate to **System Update Sets → Local Update Sets**
2. Click **New**
3. Enter Update Set Name (Example: Mandatory Field Policy)
4. Click **Submit**
5. Select **Make Current**

All changes will now be captured in this update set.

Step 2: Create UI Policy

1. Navigate to **System UI → UI Policies**

2. Click **New**
3. Select the required table (Example: Incident)
4. Provide a short description
5. Define the condition (Example: State = Resolved)
6. Save the UI Policy

Step 3: Configure UI Policy Action

1. Scroll to **UI Policy Actions**
2. Click **New**
3. Select the field to be controlled (Example: Resolution Code)
4. Set **Mandatory = True**
5. Submit the form

This configuration ensures that when the incident state becomes "Resolved," the Resolution Code field becomes mandatory.

Step 4: Testing

- Open an incident record
- Change the state to "Resolved"
- Attempt to save without entering the mandatory field
- The system should prevent saving until the field is filled

5. Project Design Phase

The design phase focuses on how the solution is structured within the platform.

System Design Overview

The solution uses ServiceNow's configuration-driven architecture:

1. **UI Policy**
    - Defines the condition under which form behavior changes.
2. **UI Policy Action**
    - Specifies which field becomes mandatory.
3. **Update Set**
    - Captures the configuration changes for migration.

Workflow Logic

User opens form → Condition is evaluated → UI Policy triggers → Field becomes mandatory → User must complete field before saving.

This design eliminates the need for complex scripting and improves maintainability.

6. Project Documentation

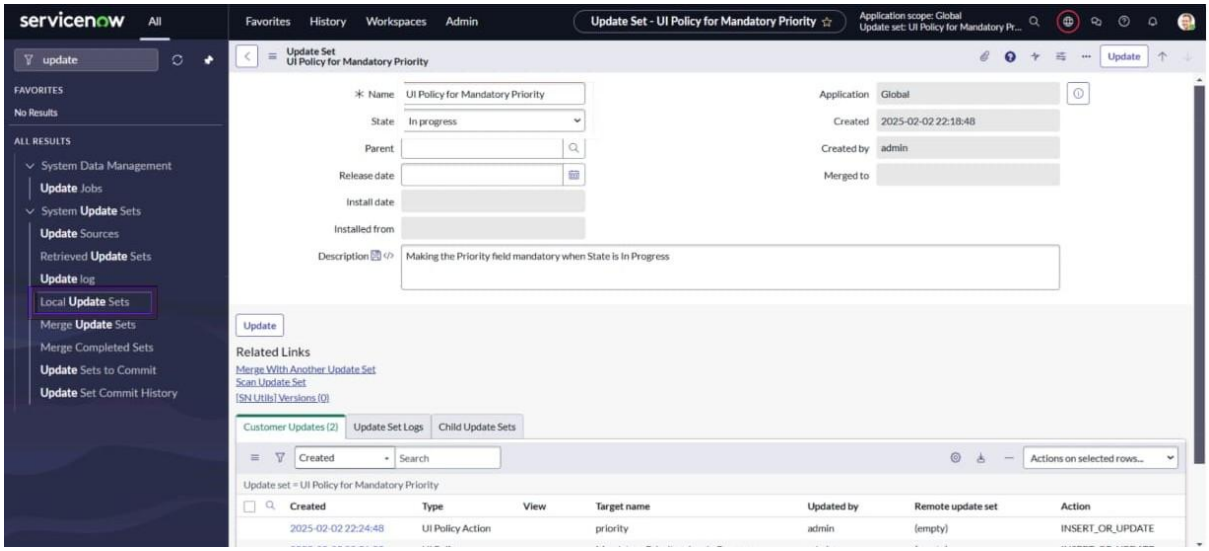
Documentation ensures that the project can be understood, maintained, and reused by other developers.

Key Documentation Elements

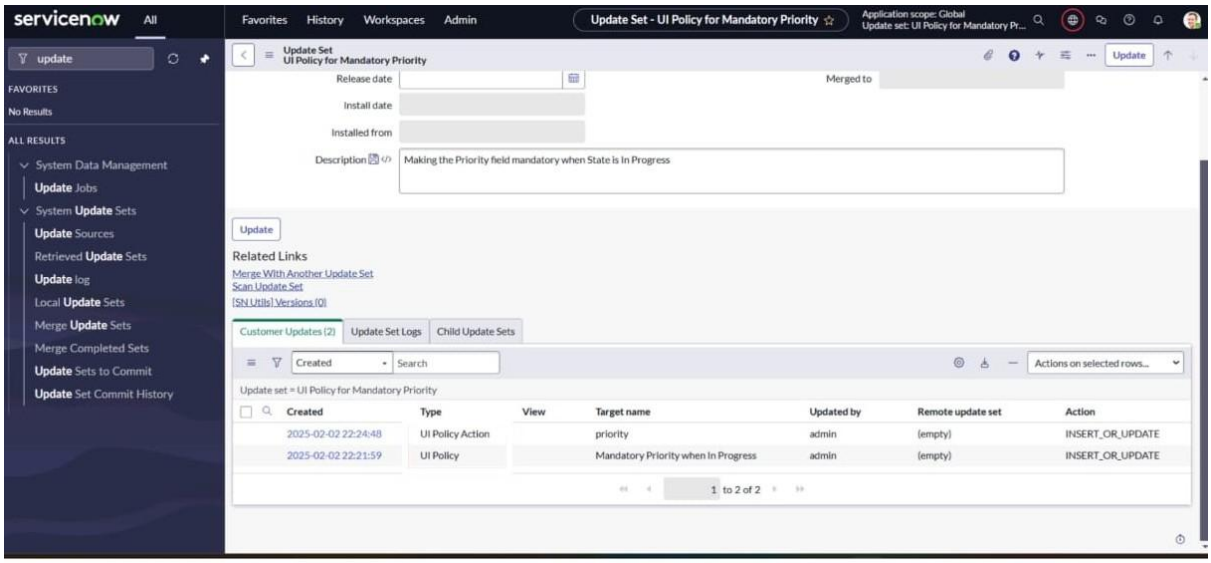
- Project Overview
- Requirement Analysis
- Configuration Steps
- Screenshots of UI Policy configuration
- Testing scenarios
- Migration process using Update Sets

Update Set Migration Documentation

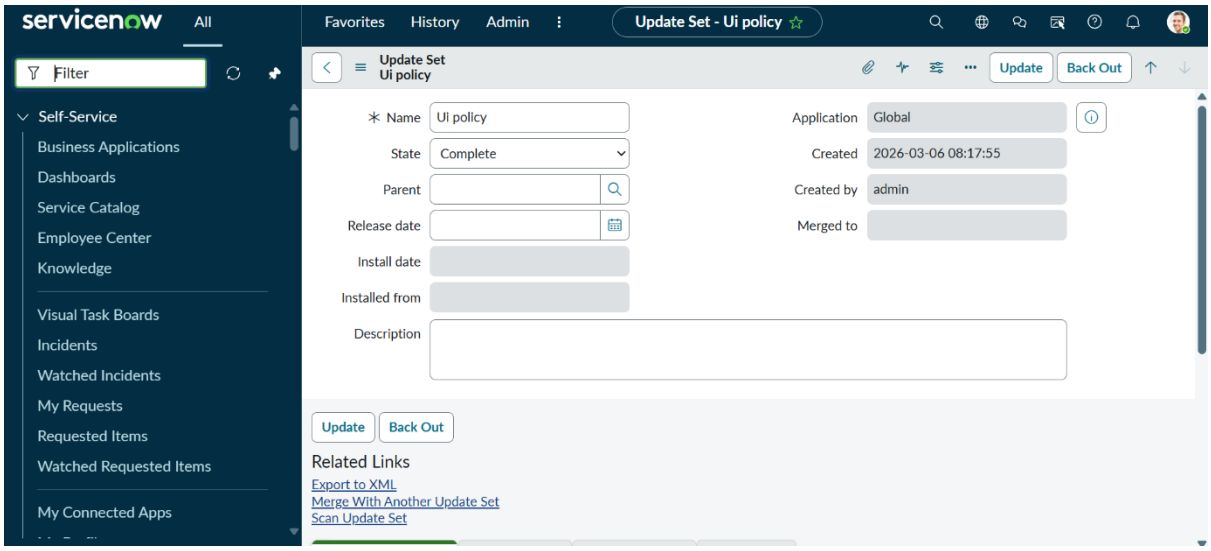
1. Open the created Update Set.



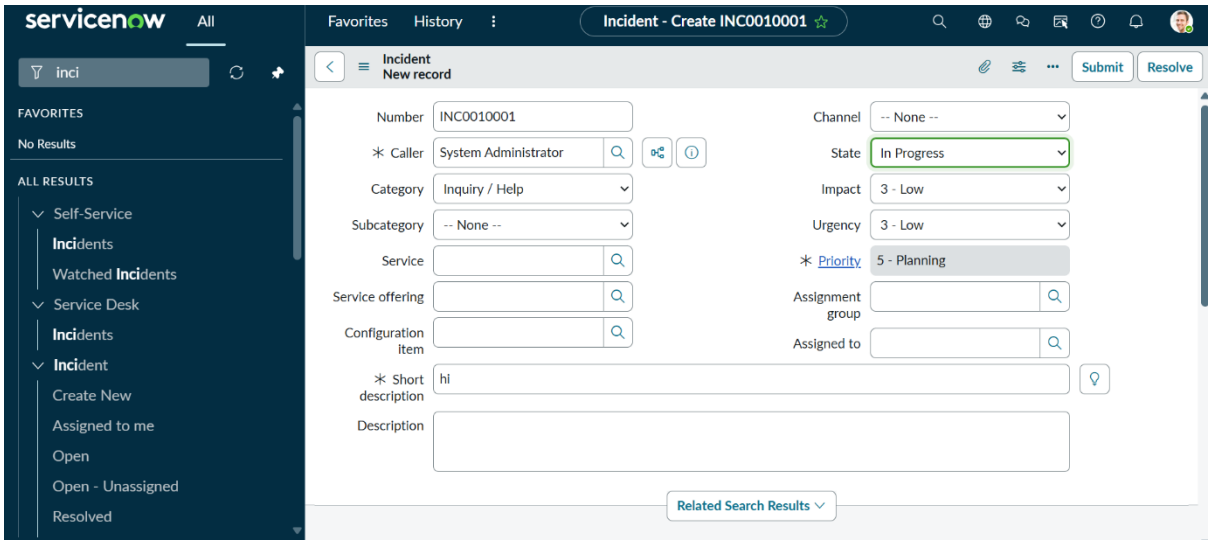
2. Verify created data under **Update set**.



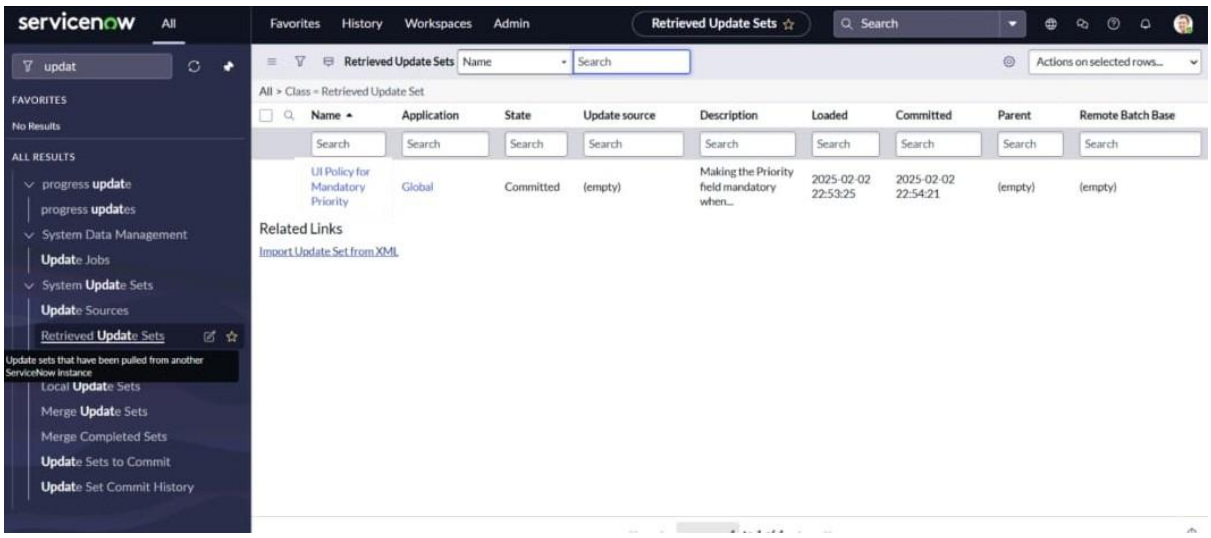
3. Export the Update Set as XML.



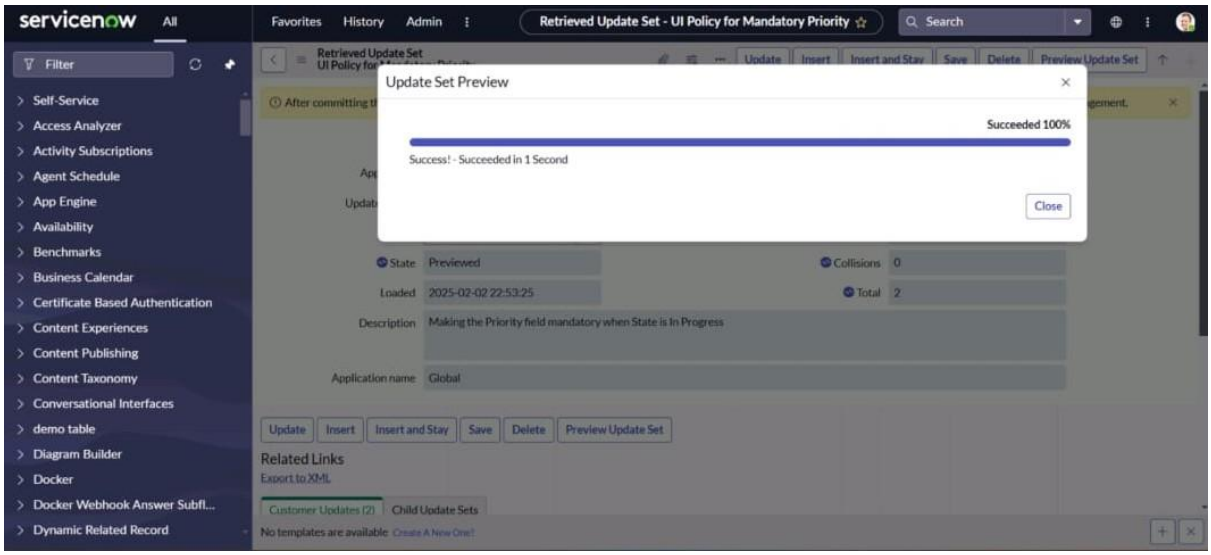
4. Import the XML file into the target instance.



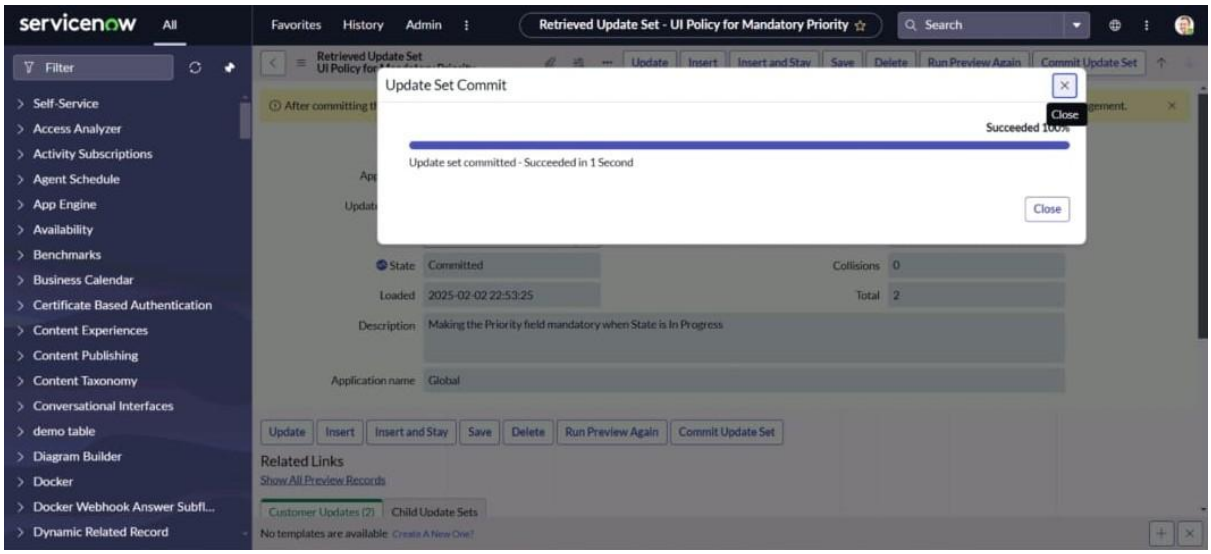
5. Open the update set that you have created in your instance.



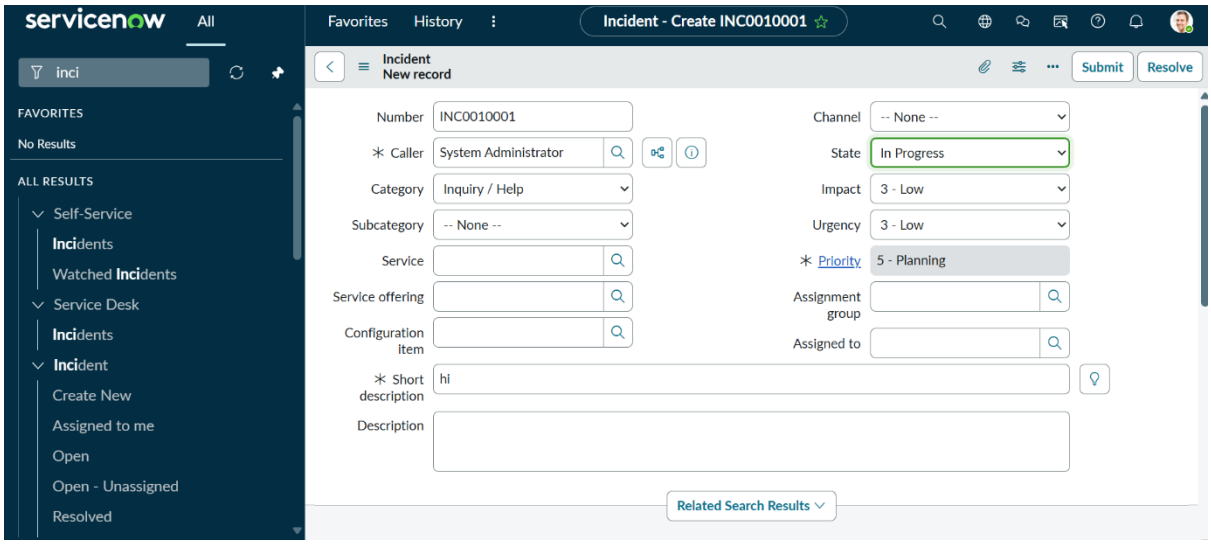
6. Preview the Update Set.



7. Commit the Update Set.



8. Testing Incident.



Proper documentation ensures smooth deployment and future maintenance.

7. Project Demonstration

The demonstration phase shows how the implemented solution works in a real environment.

Demo Scenario

Example: Incident Resolution Process

1. User opens an incident form.
2. User changes incident state to **Resolved**.
3. UI Policy triggers automatically.
4. The **Resolution Code** field becomes mandatory.
5. If the field is empty, the system prevents record submission.
6. Once the field is filled, the record can be saved successfully.

Migration Demonstration

1. Show Update Set containing the UI Policy configuration.
2. Export the Update Set from the Development instance.
3. Import it into another instance.
4. Preview and commit the Update Set.
5. Verify that the mandatory field rule works in the new instance.

Conclusion

This project demonstrates how UI Policies can be used to dynamically enforce mandatory fields in ServiceNow without writing complex scripts. It also highlights the importance of Update Sets in migrating configuration changes between instances. The implementation improves data quality, reduces manual errors, and ensures a consistent configuration management process.